جامعة الأميرة
نورة بنت عبدالرحمن
كلية علوم الحاسب والمعلومات

Data Structures

CS 212t Project

2025 – 1447

First Semester

Blood Donation & Requests Hub

3c4

Student Name	Student ID	Student E-mail
[redacted]	[redacted]	[redacted]
[redacted]	[redacted]	[redacted]
Ebtisam Bassam Alotaibi	[redacted]	[redacted]
[redacted]	[redacted]	[redacted]
[redacted]	[redacted]	[redacted]

`

I Project Introduction

1.1 Overview

Our Blood Bank Management System is made to handle the operations of blood donation centers efficiently and easily. It makes sure that the staff can quickly register new donors, find them by their ID or blood type, and remove their record if needed.Moreover, it ensures that users can submit blood requests smoothly and process them according to urgency. Also, it allows users to see all donor and request records, which helps maintain organization and accuracy. Our BBMS enables healthcare and donation professionals to offer faster and more accurate responses while simplifying donor and request management with ease.

1.2 Problem and solution

2 **The Problem:** Blood banks often face challenges in managing donor data, tracking available inventory, and ensuring a rapid response to urgent blood requests. Manual or disorganized methods can lead to delays and errors that may threaten patients' lives.

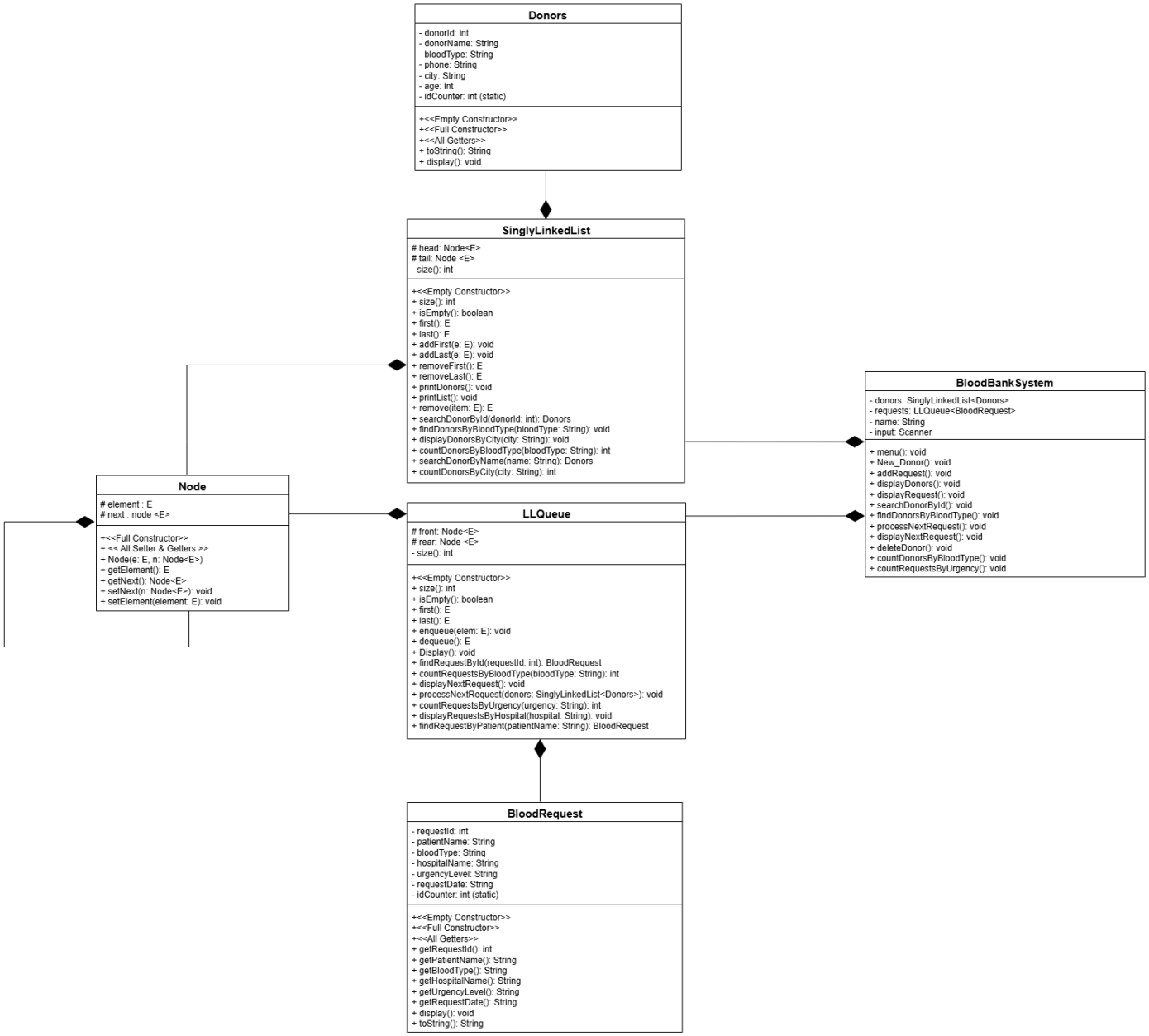
3 **The Proposed Solution:** Creating an automated system that provides a menu-driven interface for all core operations. Donor data is stored in a dynamic data structure that allows for fast addition, deletion, and searching (Singly Linked List), while blood requests are handled using a Queue structure to ensure they are processed according to the First-In, First-Out (FIFO) principle, or by priority.

3.1 Solution justification

Our Blood Bank Management System streamlines donor registration, record management, and blood request processing. This saves time and reduces human error, allowing staff to focus on urgent cases rather than manual data handling. With functions like displaying all donors, counting them by blood type, and processing requests by urgency, it allows for quick access to important data, improving the overall workflow. Overall, the system improves organization, speeds up response time, and ensures more reliable and accurate blood donation management.

`

II Project class diagram



`

III.1 Singly Linked List Methods

III.1.1 searchDonorById

This method searches for a donor using their ID

- Input: The donor ID used to search for a donor in the linked list
- Output: The method returns the Donors object if a donor with the matching ID is found

```java
    //  Search donor by ID
    public Donors searchDonorById(int donorId) {
        Node<E> current = head;

        while (current != null) {
            Donors donor = (Donors) current.element;
            if (donor.getDonorId() == donorId) {
                return donor;
            }
            current = current.getNext();
        }
        return null;
    }
```

III.1.2 findDonorsByBloodType

This method searches for donors with a specific blood type

- Input: The blood type (e.g., “A+”, “O-”, etc.) used to search for donors in the linked list
- Output: The method displays all donors that match the specified blood type. If no donor is found, it prints a message indicating that.

Function: It helps the system quickly locate all donors who have a particular blood type, which is useful for matching blood requests.

```java
    // Find donors by blood type
    public void findDonorsByBloodType(String bloodType) {
        Node<E> current = head;
        boolean found = false;

        System.out.println("---- Donors with Blood Type: " + bloodType + " ----");
        while (current != null) {
            Donors donor = (Donors) current.element;
            if (donor.getBloodType().equals(bloodType)) {
                donor.display();
                System.out.println("");
                found = true;
            }
            current = current.getNext();
        }

        if (!found) {
            System.out.println("No donors found with blood type: " + bloodType);
        }
    }
```

`

III.1.3 displayDonorsByCity

This method displays all donors from a specific city

- Input: The city name used to search for donors in the linked list
- Output: The method displays all donors who live in the specified city. If no donor is found, it prints a message indicating that.

Function: It helps the system quickly locate donors based on their city, which is useful for organizing local blood donation efforts or contacting nearby donors.

```java
    //  Display donors by city
    public void displayDonorsByCity(String city) {
        Node<E> current = head;
        boolean found = false;

        System.out.println("---- Donors in City: " + city + " ----");
        while (current != null) {
            Donors donor = (Donors) current.element;
            if (donor.getCity().equals(city)) {
                donor.display();
                System.out.println("");
                found = true;
            }
            current = current.getNext();
        }

        if (!found) {
            System.out.println("No donors found in city: " + city);
        }
    }
```

III.1.4 countDonorsByBloodType

This method counts the number of donors with a specific blood type

- Input: The blood type (e.g., “A+”, “O-”, etc.) used to search for donors in the linked list
- Output: The method returns an integer representing the total number of donors with the specified blood type

Function: It helps the system quickly know how many donors are available for a particular blood type, which is useful for planning blood donations and fulfilling blood requests.

`

```java
// Count donors by blood type
public int countDonorsByBloodType(String bloodType) {
    Node<E> current = head;
    int count = 0;

    while (current != null) {
        Donors donor = (Donors) current.element;
        if (donor.getBloodType().equals(bloodType)) {
            count++;
        }
        current = current.getNext();
    }

    return count;
}
```

III.1.5 searchDonorByName

This method searches for a donor using their name

- Input: The donor name used to search for a donor in the linked list
- Output: The method returns the Donors object if a donor with the matching name is found. If no donor matches, it returns null.

Function: It helps the system quickly locate a donor by their name, which is useful for verifying donor information or updating records.

```java
//  Search donor by name
public Donors searchDonorByName(String name) {
    Node<E> current = head;

    while (current != null) {
        Donors donor = (Donors) current.element;
        if (donor.getDonorName().equalsIgnoreCase(name)) {
            return donor;
        }
        current = current.getNext();
    }
    return null;
}
```

III.1.6 countDonorsByCity

This method counts the number of donors from a specific city

- Input: The city name used to search for donors in the linked list
- Output: The method returns an integer representing the total number of donors who live in the specified city

Function: It helps the system quickly know how many donors are available in a particular city, which is useful for organizing local blood drives and contacting nearby donors.

`

```java
    // Count donors by city
    public int countDonorsByCity(String city) {
        Node<E> current = head;
        int count = 0;

        while (current != null) {
            Donors donor = (Donors) current.element;
            if (donor.getCity().equals(city)) {
                count++;
            }
            current = current.getNext();
        }

        return count;
    }

}
```

III.2 Queue Methods

III.2.1 findRequestById

This method searches for a blood request using its request ID

- Input: The request ID used to search for a blood request in the queue
- Output: The method returns the BloodRequest object if a request with the matching ID is found. If no request matches, it returns null.

Function: It helps the system quickly locate a specific blood request, which is useful for processing urgent requests or checking request details.

```java
    // Find request by ID
    public BloodRequest findRequestById(int requestId) {

        int s = size();
        BloodRequest request = null;
        for (int i = 0; i < s; i++) {
            BloodRequest ele = (BloodRequest) dequeue();
            if (ele.getRequestId() == requestId) {
                request = ele;
            }

            enqueue((E) ele);
        }
        return request;

    }
```

`

III.2.2 countRequestsByBloodType

This method counts the number of blood requests for a specific blood type

- Input: The blood type (e.g., “A+”, “O-”, etc.) used to search for requests in the queue
- Output: The method returns an integer representing the total number of blood requests that match the specified blood type

Function: It helps the system quickly know how many requests exist for a particular blood type, which is useful for planning blood supply and prioritizing donations.

```java
// Count requests by blood type
public int countRequestsByBloodType(String bloodType) {
    if (isEmpty()) {
        return 0;
    }

    int count = 0;
    int s = size();

    for (int i = 0; i < s; i++) {
        BloodRequest request = (BloodRequest) dequeue();
        if (request.getBloodType().equals(bloodType)) {
            count++;
        }
        enqueue((E) request);
    }

    return count;
}
```

III.2.3 displayNextRequest

This method displays the next blood request in the queue

- Input: None
- Output: The method displays the details of the BloodRequest object at the front of the queue. If the queue is empty, it prints a message indicating that there are no requests.

Function: It helps the system quickly see which blood request will be processed next, which is useful for managing urgent cases and organizing blood supply efficiently.

`

```java
// Display next request
public void displayNextRequest() {
    if (isEmpty()) {
        System.out.println("No requests in the queue.");
    } else {
        BloodRequest request = (BloodRequest) first();
        System.out.println("The next request to be processed:");
        request.display();
    }
}

public void processNextRequest(SinglyLinkedList<Donors> donors) {
    if (isEmpty()) {
        System.out.println("No requests to process.");
        return;
    }

    System.out.println("--- Processing Next Request ---");
    BloodRequest request = (BloodRequest) dequeue();

    System.out.println("Request processed:");
    request.display();
    System.out.println("Searching for compatible donors...");

    donors.findDonorsByBloodType(request.getBloodType());

    System.out.println("------------------------------------");
}
```

III.2.4 countRequestsByUrgency

This method counts the number of blood requests with a specific urgency level

- Input: The urgency level (e.g., “Critical”, “High”, “Medium”) used to search for requests in the queue
- Output: The method returns an integer representing the total number of blood requests that match the specified urgency level

Function: It helps the system quickly know how many urgent or critical requests exist, which is useful for prioritizing processing and managing blood supply efficiently.

```java
//  Count requests by urgency level
public int countRequestsByUrgency(String urgency) {
    if (isEmpty()) {
        return 0;
    }

    int count = 0;
    int s = size();

    for (int i = 0; i < s; i++) {
        BloodRequest request = (BloodRequest) dequeue();
        if (request.getUrgencyLevel().equalsIgnoreCase(urgency)) {
            count++;
        }
        enqueue((E) request);
    }

    return count;
}
```

III.2.5 displayRequestsByHospital

This method displays the next blood request in the queue

- Input: None
- Output: The method displays the details of the BloodRequest object at the front of the queue. If the queue is empty, it prints a message indicating that there are no requests.

`

Function: It helps the system quickly see which blood request will be processed next, which is useful for managing urgent cases and organizing blood supply efficiently.

```java
// Display next request
public void displayNextRequest() {
    if (isEmpty()) {
        System.out.println("No requests in the queue.");
    } else {
        BloodRequest request = (BloodRequest) first();
        System.out.println("The next request to be processed:");
        request.display();
    }
}

public void processNextRequest(SinglyLinkedList<Donors> donors) {
    if (isEmpty()) {
        System.out.println("No requests to process.");
        return;
    }

    System.out.println("--- Processing Next Request ---");
    BloodRequest request = (BloodRequest) dequeue();

    System.out.println("Request processed:");
    request.display();
    System.out.println("Searching for compatible donors...");

    donors.findDonorsByBloodType(request.getBloodType());

    System.out.println("-----------------------------------");
}
```

III.2.6 findRequestByPatient

This method searches for a blood request using the patient's name

- Input: The patient name used to search for a blood request in the queue
- Output: The method returns the BloodRequest object if a request with the matching patient name is found. If no request matches, it returns null.

Function: It helps the system quickly locate a blood request for a specific patient, which is useful for tracking urgent needs and managing blood request fulfillment efficiently.

```java
//  Find request by patient name
public BloodRequest findRequestByPatient(String patientName) {

    int s = size();
    BloodRequest request = null;
    for (int i = 0; i < s; i++) {
        BloodRequest ele = (BloodRequest) dequeue();
        if (ele.getPatientName().equals(patientName)) {
            request = ele;
        }

        enqueue((E) ele);
    }
    return request;

}
```

`

IV Main Class

1) **Scanner Initialization Reads user input from the keyboar**

```
public static void main(String[] args) {

    Scanner input = new Scanner(System.in);
    BloodBankSystem bds = new BloodBankSystem("Blood Donation & Requests Hub");
```

2) **Create BloodBankSystem Object the Purpose: Initializes the system with a name, a linked list for donors, and a queue for blood requests.**

```
public class Main {

    public static void main(String[] args) {

        Scanner input = new Scanner(System.in);
        BloodBankSystem bds = new BloodBankSystem("Blood Donation & Requests Hub");

        int option = 0;
        do {
```

3) **Start Menu Loop the Purpose: Continuously displays the menu and allows the user to select options until they choose to exit.**

```
int option = 0;
do {
    try {


        bds.menu();
        System.out.print("Enter your choice: ");
        option = input.nextInt();
```

4)Option 1 – Register New Donor the Purpose: Adds a new donor to the donor list.

```
case 1:
    bds.New_Donor();
    System.out.println("");
    break;
```

`

5)Option 2 – Submit Blood Request the Purpose: Adds a new blood request to the queue.

```
case 2:
    bds.addRequest();
    System.out.println("");
    break;
```

6)Option 3 – Display All Donors the Purpose: Shows all registered donors in the system.

```
case 3:
    bds.displayDonors();
    System.out.println("");
    break;
```

7)Option 4 – Display All Request the Purpose: Displays all pending blood requests.

```
case 4:
    bds.displayRequest();
    System.out.println("");
    break;
```

8)Option 5 – Remove Donor the Purpose: Deletes a donor by their ID.

```
case 5:
    bds.deleteDonor();
    System.out.println("");
    break;
```

`

9)Option 6 – Find Donor by Blood Type the Purpose: Searches for donors with specific blood type.

```
case 6:
    bds.findDonorsByBloodType();
    System.out.println("");
    break;
```

10)Option 7 – Count Donors by Blood Type The Purpose: Counts how many donors have each blood type.

```
case 7:

    bds.countDonorsByBloodType();
    System.out.println("");
    break;
```

11)Option 8 – Count Blood Requests by Urgency The purpose: Counts blood requests by urgency level.

```
case 8:
    bds.countRequestsByUrgency();
    System.out.println("");
    break;
```

12) Option 9 – Process Next Blood Request The Purpose: Processes the next request and matches it with donors.

```
case 9:

    bds.processNextRequest();
    System.out.println("");
    break;
```

13) Option 10 – Exit Program The Purpose: Ends the program with a thank-you message.

```
case 10:
    System.out.println("\nThank you for using Blood Donation System!");
    System.out.println("");
    break;
```

`

14)Default Case – Invalid Option The Purpose: Handles wrong input and displays an error message.

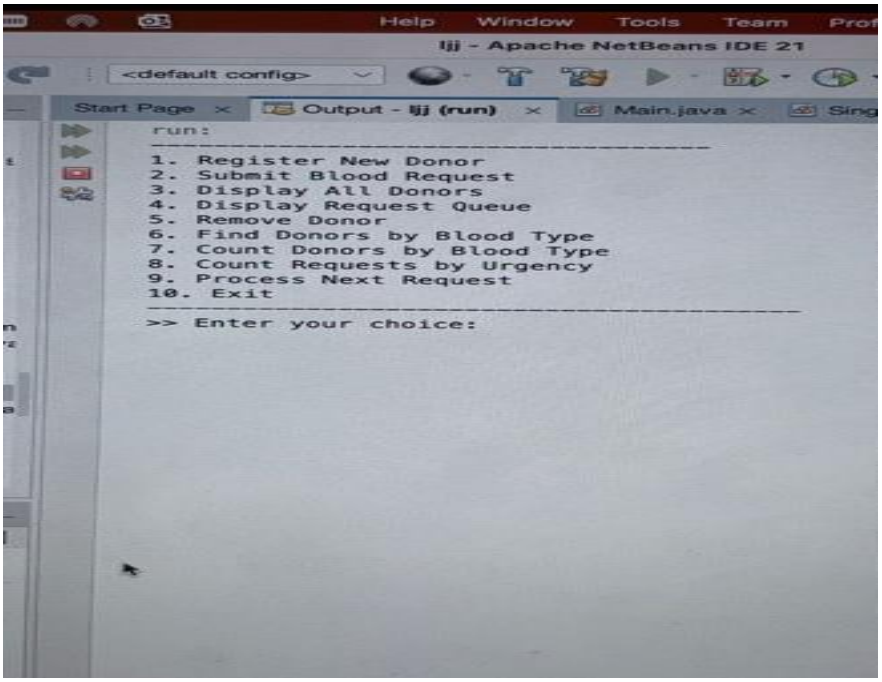
```
  default:
      System.out.println("Invalid option!");
```

15) Exception Handling – The Purpose: Prevents program crash when user enters wrong input type.

```
} catch (InputMismatchException e) {
    System.err.println("Invalid input");
    input.next();
}
```

16) End of Loop Condition – The Purpose: Repeats the menu until user selects option 10.

```
      }
  } while (option != 10);
```



`

```
10. Exit
-----------------------------------------
>> Enter your choice: 1
 New Donor
Enter Age: 12
Enter Donors Name: sara
Enter Blood Type (A+, A-, B+, B-, O+, O-, AB+, AB-): A-
Enter Phone Number: 9876987621
Enter City: riyadh

Donor added successfully!
Donor ID: 1001
Name: sara
Blood Type: A-
Phone: 9876987621
City: riyadh
Age: 12

-----------------------------------
```

```
7. Count Donors by Blood Type
8. Count Requests by Urgency
9. Process Next Request
10. Exit
-----------------------------------------
>> Enter your choice: 2

 create Blood Request
Enter Patient Name: sara
Enter Blood Type Needed: B+
Enter Hospital Name: alhabib
Enter Urgency Level (Critical/High/Medium): High
Enter Request Date (YYYY-MM-DD): 2025/10/05
Blood request submitted successfully!
Request ID: 2001
Patient Name: sara
Blood Type: B+
Hospital: alhabib
Urgency: High
Date: 2025/10/05
-----------------------------------

-----------------------------------
1. Register New Donor
2. Submit Blood Request
3. Display All Donors
4. Display Request Queue
5. Remove Donor
6. Find Donors by Blood Type
7. Count Donors by Blood Type
8. Count Requests by Urgency
9. Process Next Request
10. Exit
-----------------------------------------
```

```
3. Display All Donors
4. Display Request Queue
5. Remove Donor
6. Find Donors by Blood Type
7. Count Donors by Blood Type
8. Count Requests by Urgency
9. Process Next Request
10. Exit
-----------------------------------------
>> Enter your choice: 3

ALL DONORS
Donor ID: 1001
Name: sara
Blood Type: A-
Phone: 9876987621
City: riyadh
Age: 12

-----------------------------------
1. Register New Donor
2. Submit Blood Request
3. Display All Donors
4. Display Request Queue
5. Remove Donor
6. Find Donors by Blood Type
7. Count Donors by Blood Type
8. Count Requests by Urgency
9. Process Next Request
10. Exit
-----------------------------------------
>> Enter your choice: |
ljj (run)  running...
```

`

```
1. Register New Donor
2. Submit Blood Request
3. Display All Donors
4. Display Request Queue
5. Remove Donor
6. Find Donors by Blood Type
7. Count Donors by Blood Type
8. Count Requests by Urgency
9. Process Next Request
10. Exit
----------------------------------------
>> Enter your choice: 4

 ALL REQUEST
Request ID: 2001 | Patient: sara | Blood Type: B+ | Hospital: alhabib | Urgency: High | Date: 2025/10/05

----------------------------------
1. Register New Donor
2. Submit Blood Request
3. Display All Donors
4. Display Request Queue
5. Remove Donor
6. Find Donors by Blood Type
7. Count Donors by Blood Type
8. Count Requests by Urgency
9. Process Next Request
10. Exit
----------------------------------------
>> Enter your choice:
```

```
9. Process Next Request
10. Exit
----------------------------------------
>> Enter your choice: 5

Enter the ID of the donor you wish to remove: 1001

Donor information removed successfully!
Removed donor name: qwed

-----------------------------------
1. Register New Donor
2. Submit Blood Request
3. Display All Donors
4. Display Request Queue
5. Remove Donor
6. Find Donors by Blood Type
7. Count Donors by Blood Type
8. Count Requests by Urgency
9. Process Next Request
10. Exit
----------------------------------------
>> Enter your choice:
```

```
9. Process Next Request
10. Exit
----------------------------------------
>> Enter your choice: 6
Enter Blood Type (A+, A-, B+, B-, O+, O-, AB+, AB-): A
---- Donors with Blood Type: A ----
No donors found with blood type: A

-----------------------------
1. Register New Donor
2. Submit Blood Request
3. Display All Donors
4. Display Request Queue
5. Remove Donor
6. Find Donors by Blood Type
7. Count Donors by Blood Type
8. Count Requests by Urgency
9. Process Next Request
10. Exit
----------------------------------------
>> Enter your choice:
```

`

```
7. Count Donors by Blood Type
8. Count Requests by Urgency
9. Process Next Request
10. Exit
----------------------------------------
>> Enter your choice: 7

Please enter the blood type to check: Total donors registered with blood type : 0

----------------------------------
1. Register New Donor
2. Submit Blood Request
3. Display All Donors
4. Display Request Queue
5. Remove Donor
6. Find Donors by Blood Type
7. Count Donors by Blood Type
8. Count Requests by Urgency
9. Process Next Request
10. Exit
----------------------------------------
>> Enter your choice:
```

```
----------------------------------------
>> Enter your choice: 8

Please specify the urgency level (Critical / High / Medium): High

Total number of requests marked as High: 0

-----------------------------------
1. Register New Donor
2. Submit Blood Request
3. Display All Donors
4. Display Request Queue
5. Remove Donor
6. Find Donors by Blood Type
7. Count Donors by Blood Type
8. Count Requests by Urgency
9. Process Next Request
10. Exit
----------------------------------------
>> Enter your choice:
```

`

```
10. Exit
----------------------------------------
>> Enter your choice: 9
--- Processing Next Request ---
Request processed:
Request ID: 2001
Patient Name: sd
Blood Type: rscf
Hospital: sf
Urgency: a
Date: y
Searching for compatible donors...
---- Donors with Blood Type: rscf ----
No donors found with blood type: rscf
-------------------------------------

-----------------------------------
1. Register New Donor
2. Submit Blood Request
3. Display All Donors
4. Display Request Queue
5. Remove Donor
6. Find Donors by Blood Type
7. Count Donors by Blood Type
8. Count Requests by Urgency
9. Process Next Request
```

```
2. Submit Blood Request
3. Display All Donors
4. Display Request Queue
5. Remove Donor
6. Find Donors by Blood Type
7. Count Donors by Blood Type
8. Count Requests by Urgency
9. Process Next Request
10. Exit
----------------------------------------
>> Enter your choice: 10

Thank you for using Blood Donation System!

-----------------------------------
1. Register New Donor
2. Submit Blood Request
3. Display All Donors
4. Display Request Queue
5. Remove Donor
6. Find Donors by Blood Type
7. Count Donors by Blood Type
8. Count Requests by Urgency
9. Process Next Request
10. Exit
----------------------------------------
>> Enter your choice:
```

`

V Work distribution

	[redacted]	[redacted]	[redacted]	Ebtisam Bassam Alotaibi	[redacted]
Singly Linked List	✓	✓	✓	✓	✓
Singly Linked List searchDonorById	✓				
Singly Linked List findDonorsByBloodType		✓			
Singly Linked List displayDonorsByCity			✓		
Singly Linked List countDonorsByBloodType				✓	
Singly Linked List searchDonorByName					✓
Singly Linked List countDonorsByCity	✓	✓	✓	✓	✓
Queue	✓	✓	✓	✓	✓
Queue findRequestById	✓				
Queue countRequestsByBloodType		✓			
Queue displayNextRequest			✓		
Queue countRequestsByUrgency				✓	
Queue displayRequestsByHospital					✓
Queue findRequestByPatient	✓	✓	✓	✓	✓